

Rekursion: SumList:Generic

Exkurs: Was wäre, wenn auch der Rückgabetyt ein Generic `T` wäre?

Das ist in Java **nicht trivial**, da Java aufgrund der **Type Erasure** (Typ-Löschung) zur Laufzeit nicht weiß, was `T` eigentlich ist.

```
T result = element1 + element2; // COMPILER-FEHLER!
```

Der Compiler weiß nicht, ob `T` ein `Integer`, ein `String` oder ein `Hund` ist. Der `+` Operator ist in Java nicht für beliebige Generics definiert.

<https://docs.oracle.com/javase/specs/jls/se8/html/jls-4.html#jls-4.6>

Um eine generische Addition zu ermöglichen, musst du der Methode von außen beibringen, **wie** addiert wird und was der **Startwert** (die "Null") ist. Dafür nutzen wir `BinaryOperator<T>`. (Funktionale Interfaces und Lambda ftw!)

<https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/util/function/BinaryOperator.html>

```
public class SumListGenericT {  
  
    // Wir brauchen:  
    // 1. Die Liste  
    // 2. Das "Null"-Element da wir nicht wissen, ob 0, 0.0, 0f gemeint ist  
    // 3. Eine Funktion, die weiß, wie man zwei T's addiert  
    public static <T> T sumGeneric(List<T> list, T zeroElement,  
        BinaryOperator<T> adder) {  
        if (list.isEmpty()) {  
            return zeroElement;  
        }  
  
        T head = list.get(0);  
        List<T> tail = list.subList(1, list.size());  
  
        // Rekursion: Addiere head mit dem Ergebnis des Rests  
        // adder.apply() ersetzt hier das "+"  
        return adder.apply(head, sumGeneric(tail, zeroElement, adder));  
    }  
  
    static void main(String[] args) {  
        // Beispiel für Integer  
        List<Integer> ints = List.of(1, 2, 3);  
        // Wir übergeben: Liste, Startwert 0, und eine Lambda-Funktion für (a + b)  
        Integer sumInt = sumGeneric(ints, 0, (a, b) -> a + b);  
        System.out.println("Integer Summe: " + sumInt);  
  
        // Beispiel für Strings (Konkatenation statt Mathe!)  
        List<String> strings = List.of("Hallo", " ", "Welt");  
        String sumString = sumGeneric(strings, "", (a, b) -> a + b);  
        System.out.println("String Summe: " + sumString);  
    }  
}
```

Also: Es geht nicht direkt. Du musst die Logik für die Addition `((a, b) -> a + b)` immer mitliefern!